MOD -03 -> Advanced Java

LUMINAR
TECHNOLAB

String

LUMINAR
TECHNOLAB

Strings, String Buffer, String Builder and String Tokenizer

STRING

1. Java string is basically an object that represents sequence of char values.
2. An array of characters works same as java string.

For example:

```
char[] ch={'l','u','m','i','n','a','r'};
    String s=new String(ch);
```

is same as:

```
String s="luminar";
```

```
public class StringExample {
      public static void main(String args[]){
            String s1="java";//creating string by java string literal
            char ch[]={'s','t','r','i','n','g','s'};
            String s2=new String(ch);//converting char array to string
            String s3=new String("example");//creating java string by new
keyword
            System.out.println(s1);
            System.out.println(s2);
            System.out.println(s3);

      }
```

What is Heap memory and String Constant Pool in Java?

Constant Pool It is a special memory area in Heap

IMMUTABLE (CANNOT BE CHANGED)

1. The java String is immutable -> All String objects, regardless of where they are created(String Constant Pool (SCP) or Heap memory), are immutable. ie. The String class itself is designed to be immutable. This immutability is an architectural feature of the class, not a property tied to a specific memory location.

There are two ways to create string object in Java.

1. Using string literal

Example: String s1 = “java”;

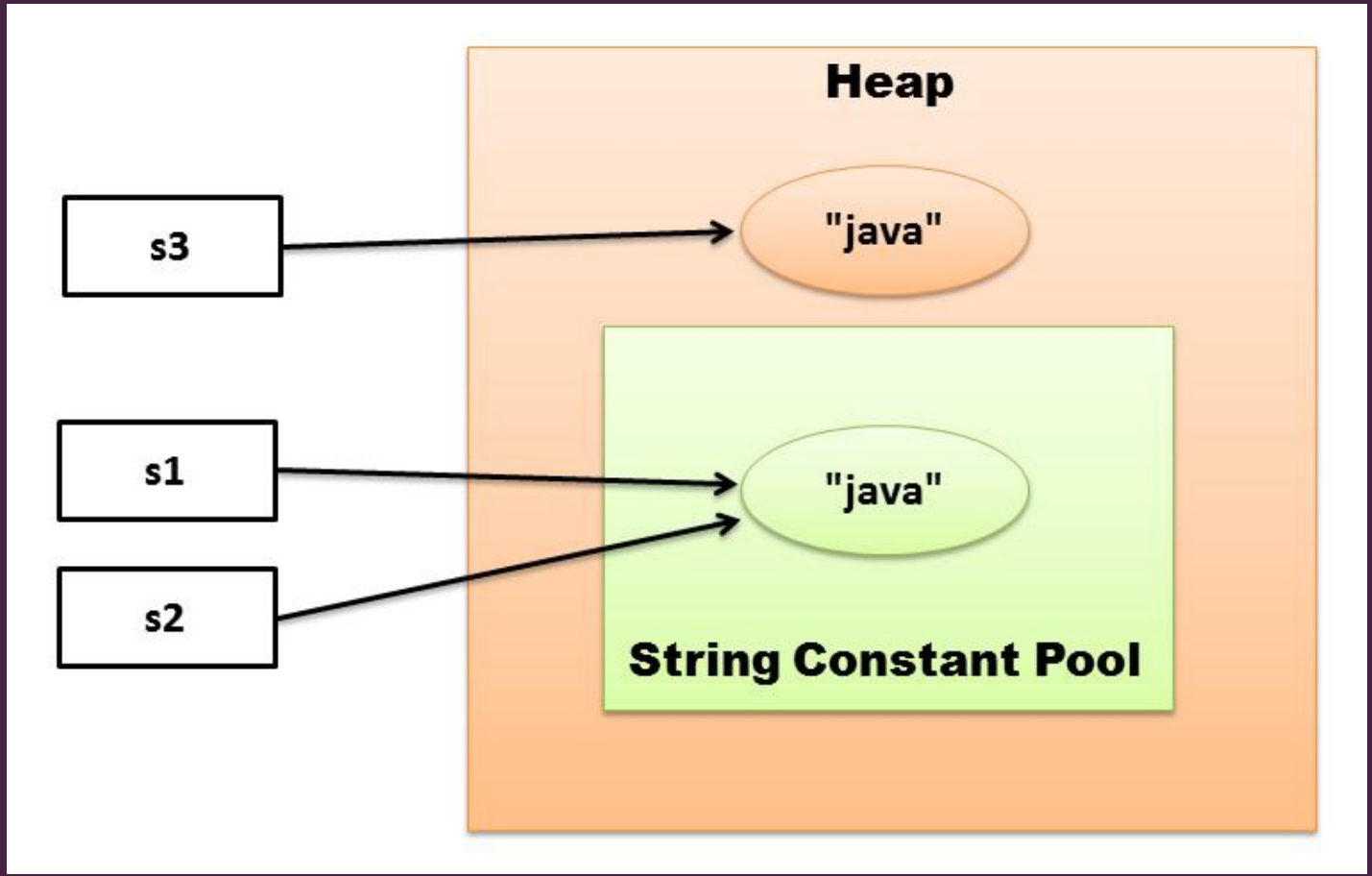
A string written inside double quotes is called string literal.

Whenever we create string literal the JVM checks string constant pool. Strings are represented in UTF-16 format.

2. Using new keyword

Example: String s3= new String("java");

When we create string using new keyword it goes to heap.



String is immutable and final in Java and every modification in String creates a new String object.

For example, when you convert uppercase String to lowercase, a new String is created. Even when you remove space by calling the trim() method, a new String is returned. Since String is an immutable class every time a new String is created and the older one is discarded which creates lots of temporary garbage in the heap.

Why string objects are immutable in java?

1. The primary reason String objects are immutable in Java is to ensure security, thread safety, efficiency and hashing.

Security

String objects are frequently used as parameters for security-critical methods, such as file paths, network connection addresses, and database connection strings.

If a String were mutable, an attacker could pass a legitimate value to a security-check method, and then change the String's content after the check was performed.

Immutability guarantees that once a security check passes, the underlying value used for that check cannot be altered.

Thread Safety

Immutable objects are inherently thread-safe.

Because a String's value can never change after creation, multiple threads can safely share and read the same String object without needing explicit synchronization or locks. This simplifies concurrent programming and prevents data corruption in multi-threaded environments.

Efficiency (String Constant Pool)

Immutability enables the String Constant Pool (SCP), a memory optimization technique.

The JVM stores String literals in the SCP and allows multiple reference variables to point to the same single object to save memory.

This sharing is only possible because the object is immutable. If one reference could modify the shared String, it would accidentally change the value for all other references pointing to it.

Hash Code Caching

String objects are heavily used as keys in Hash collections (like HashMap).

A String's hash code is calculated only once during its creation and then cached. Since the content never changes, the hash code also never changes. This ensures fast and reliable retrieval from hash-based collections.

Detailed Explanation

String Immutability Comes from the String Class Design, Not Its Location in Memory

1. String is final

```
public final class String {
   private final char value[];
   ...
}
```

The String class in Java is marked as final(— it cannot be subclassed), and all its internal state (char[] value) is private and final.

So any object of String, no matter how it's created, is read-only after creation.

```
String s = new String("java");
s.concat("world");   // doesn't change the original string
```

It returns a new String object instead. So s still points to "java".
In Java, objects don’t have names, only references (variable names) do.

```
String result = s.concat("world");
```

Then the reference variable result refers to the new object "javaworld".But if you just write:

```
s.concat("world");
```

Then the object is created in memory, but no reference points to it, so Java's garbage collector may later clean it up.

```
String s = new String("java");
s.concat("world");   // Does nothing
System.out.println(s);   // prints: java
```

Because s.concat("world") returns a new string, but you didn't store it.

What Happens in Memory in the case of String literal

```
String s1 = "Hello";
s1 = s1 + " World";
```

Step 1:

```
String s1 = "Hello";
```

Java checks if "Hello" already exists in the String Pool (a special area in heap memory).If not, it creates a new object and stores the reference in s1.

Step 2

```
s1 = s1 + " World";
```

"Hello" cannot be changed, so Java:

1. Creates a new String object: "Hello World“
2. Stores it in the String Pool (if not already there)
3. s1 now points to "Hello World"
4. the original "Hello" string still exists (if no other reference, it'll be garbage collected).

	String Literal	new Operator
	String s = "hello";	String s = new String("hello");
Memory Location	String Constant Pool (SCP), which is part of the Heap.	Heap memory (outside the SCP).
Object Creation	Checks the SCP first. If the value "hello" exists, it returns a reference to the existing object. If not, it creates a new object in the SCP and then returns its reference.	Always creates a new object in the general Heap space, regardless of whether the value already exists in the SCP.

So, now the big question is how do you manipulate String in Java without creating String garbage? (OR) For mutable string, you can use StringBuffer and StringBuilder classes.

- StringBuffer is old class
- StringBuilder is newly added in Java 5

StringBuffer	StringBuilder
Thread safe	Not thread safe
Synchronized	Not synchronized
Slower than StringBuilder	Faster than StringBuffer

If you see StringBuilder vs StringBuffer you will find that they are exactly similar and all API methods applicable to StringBuffer are also applicable to StringBuilder in Java.

Even though all StringBuffer, StringBuilder, and String are from same type hierarchy i.e. they extend from CharSequence interface, you cannot cast StringBuilder to StirngBuffer or StringBuilder to String in Java. It will throw java.lang.ClassCastException, if you tried to cast even StringBuffer to String in Java.

String important methods

1. Java String compare

We can compare string in java on the basis of content and reference.

There are three ways to compare string in java:

1. It is used in authentication (by equals() method)
2. sorting (by compareTo() method)
3. reference matching (by == operator)

1) String compare by equals() method

The String equals() method compares the original content of the string. It compares values of string for equality. String class provides two methods:

1. public boolean equals(Object another) compares this string to the specified object.
2. public boolean equalsIgnoreCase(String another) compares this String to another string, ignoring case.

String class overrides equals() and hashcode() method and two Strings are considered to be equal if they contain exactly same character in same order and in same case. If you want to ignore case comparison of two strings consider using equalsIgnoreCase() method.

```
public class Teststringcomparison {
    public static void main(String args[]){
        String s1=“Luminar";
        String s2=“Luminar";
        String s3=new String(“Luminar");
        String s4=“LuminarT";
        System.out.println(s1.equals(s2));//true
        System.out.println(s1.equals(s3));//true
        System.out.println(s1.equals(s4));//false
    } }
```

```
public class Teststringcomparison {
    public static void main(String args[]){
        String s1=“Luminar";
        String s2=“LUMINAR";

        System.out.println(s1.equals(s2));//false
        System.out.println(s1.equalsIgnoreCase(s2));//true
      }

}
```

2) String compare by == operator

The = = operator compares references not values.

When you compare two String literals using equality operator "==" it returns true because they are the actually same instance of String. Anyway comparing an object with equality operator is bad practice in Java and you should always use the equals method to check equality.

```
public class Teststringcomparison {
    public static void main(String args[]){
        String s1=“Luminar";
        String s2="Luminar";

        String s3=new String("Luminar");
        System.out.println(s1==s2);//true (because both refer to same instance)
        System.out.println(s1==s3);//false(because s3 refers to instance created in nonpool)
      }

}
```

3) String compare by compareTo() method

The String compareTo() method compares values lexicographically and returns an integer value that describes if first string is less than, equal to or greater than second string.

Suppose s1 and s2 are two string variables. If:

- s1 == s2 :0
- s1 > s2 :positive value
- s1 < s2 :negative value

```
public class Teststringcomparison {
    public static void main(String args[]){
        String s1=“Luminar";
        String s2= Luminar ";

        String s3=new String("Oracle");
        System.out.println(s1.compareTo(s2));//0
        System.out.println(s1.compareTo(s3));//- value(because s1<s3)
        System.out.println(s3.compareTo(s1));//+value(because s3 > s1 )
      }

}
```

String Concatenation in Java

In java, string concatenation forms a new string that is the combination of multiple strings. There are two ways to concat string in java:

1. By + (string concatenation) operator
2. By concat() method

The "+" operator is overloaded for String and used to concatenated two string. Internally "+" operation is implemented using either StringBuffer or StringBuilder.

1) String Concatenation by + (string concatenation) operator

```
public class TestConcat {
    public static void main(String args[]){
        String s=“Luminar"+" Technolab";
        System.out.println(s);
    }
}
```

2) String Concatenation by concat() method

```
public class TestConcat {
    public static void main(String args[]){
            String s1=“Luminar ";
        String s2=“Technolab";
        String s3=s1.concat(s2);
        System.out.println(s3);
      }
```

Substring in Java

A part of string is called substring. In other words, substring is a subset of another string. In case of substring startIndex is inclusive and endIndex is exclusive.

```
public class TestSubstring {
    public static void main(String args[]){
        String s="LuminarTechnolab";
        System.out.println(s.substring(7));//  Technolab
        System.out.println(s.substring(0,7));//  Luminar
      }  }
```

Java String class methods

```
public class JavaStrings {
    public static void main(String args[]) {
        String s=“Luminar";
        System.out.println(s.toUpperCase());
        System.out.println(s.toLowerCase());
        System.out.println(s.trim()); //delete left and right blank spaces
        System.out.println(s.startsWith(“Lu"));//true
        System.out.println(s.endsWith(“r"));//true
        System.out.println(s.charAt(0));
        System.out.println(s.charAt(3));
        System.out.println(s.length());
        String replaceString=s.replace(“ar",“art");
        System.out.println(replaceString);
```

Java String intern() method

You can manually force a String object (even one created with new) into the SCP using the intern() method

When you call s.intern() on a String object s, the Java Virtual Machine (JVM) performs the following check:

Checks the SCP: It searches the String Constant Pool (SCP) for a String object that has the same content (the same sequence of characters) as the object s.

Scenario 1: Match Found

If a matching String object is already present in the SCP, the intern() method simply returns the reference to that pooled object.

Scenario 2: No Match Found

If no such String object is found in the SCP, the intern() method adds a copy of the original String object s to the pool. It then returns the reference to this newly pooled object.

The result of intern() is always a reference to a String object guaranteed to be residing within the SCP.

```
String s1 = new String("Luminar"); // Object created in the Heap
String s2 = "Luminar";             // Object created in the SCP

// s1 == s2 is FALSE (Different memory addresses)

String s3 = s1.intern(); // s3 now points to the "Luminar" object in the SCP

// s3 == s2 is TRUE (Both point to the same object in the SCP)
```

Java String valueOf() method

The string valueOf() method coverts given type such as int, long, float, double, boolean, char and char array into string.

```
public class JavaStrings {
    public static void main(String args[]) {

        int a=10;
        String s=String.valueOf(a);
        System.out.println(s+10);
    }
}
```

Java String replace() method

```
public class JavaStrings {
      public static void main(String args[]) {

            String s1="Java is a programming language. Java is a platform.
Java is Platform independent.";
            String replaceString=s1.replace("Java","Android");//replaces all
occurrences of "Java" to "Android"
            System.out.println(replaceString);
        }
}
```

Java toString() method

1. If you want to represent any object as a string, toString() method comes into existence.
2. The toString() method returns the string representation of the object.
3. If you print any object, java compiler internally invokes the toString() method on the object. So overriding the toString() method, returns the desired output, it can be the state of an object etc. depends on your implementation.

Advantage of Java toString() method

1. By overriding the toString() method of the Object class, we can return values of the object, so we don't need to write much code.

Important Constructors of StringBuffer class

Constructor	Description
StringBuffer()	creates an empty string buffer with the initial capacity of 16.
StringBuffer(String str)	creates a string buffer with the specified string.
StringBuffer(int capacity)	creates an empty string buffer with the specified capacity as length.

StringBuffer and StringBuilder -> Capacity

```
public class StringBufferBuilderCapacity {
      public static void main(String[] args) {

            StringBuffer sb = new StringBuffer(6); // capacity=6
            System.out.println("Initial capacity : " + sb.capacity());
            sb.append("Merit Campus"); // length=12
            System.out.println("Lenght of sb : " + sb.length());
// string is greather than 6. So new capacity is (6*2)+2=14
//capacity is > length . So capacity is considered as 14
            System.out.println("StringBuffer capacity : " + sb.capacity());

      }

}
```

```
public class StringBufferBuilderCapacity {
    public static void main(String[] args) {

        StringBuffer sb = new StringBuffer();
        System.out.println("Intial capacity : " + sb.capacity());//default=16
        sb.append("Luminar Technolab Ernakulam Kakkanad Kochi");
System.out.println("Length of sb : " + sb.length());
// string is greather than 6. So new capacity is (16*2)+2=34
//capacity is < length . So capacity is considered as string length

        System.out.println("StringBuffer capacity : " + sb.capacity());

  }

}
```

StringTokenizer in Java

1. The java.util.StringTokenizer class allows you to break a string into tokens. It is simple way to break string.

Constructors of StringTokenizer class

1. There are important constructors defined in the StringTokenizer class.

Constructor	Description
StringTokenizer(String str)	creates StringTokenizer with specified string.
StringTokenizer(String str, String delim)	creates StringTokenizer with specified string and delimeter.

Methods of StringTokenizer class

The 6 useful methods of StringTokenizer class are as follows:

Public method	Description
boolean hasMoreTokens()	checks if there is more tokens available.
String nextToken()	returns the next token from the StringTokenizer object.
String nextToken(String delim)	returns the next token based on the delimeter.
boolean hasMoreElements()	same a hasMoreTokens() method.
Object nextElement()	same as nextToken() but its return type is Object.
int countTokens()	returns the total number of tokens.

```
import java.util.StringTokenizer;

public class StringToken {
   public static void main(String args[]){
         StringTokenizer st = new StringTokenizer("Luminar Technolab
Kakkanad Kochi"," ");
           while (st.hasMoreTokens()) {
               System.out.println(st.nextToken());
           }
   }
```

```
import java.util.StringTokenizer;

public class StringToken {
   public static void main(String args[]){
         StringTokenizer st = new
StringTokenizer("Luminar:Technolab:Kakkanad:Kochi");
         // printing next token
            System.out.println("Next token is : " + st.nextToken(":"));
   }

}
```

Thank you ☺ Happy learning ☺